

Entornos de desarrollo

UNIDAD 5

5. Elaboración de diagramas de clases

Elaboración de diagramas de clases

Objetivos

- Aprender los conceptos básicos de la orientación a objetos.
- Comprender los fundamentos del lenguaje UML.
- Conocer los tipos de diagramas que se pueden construir en el lenguaje UML.
- Saber cómo representar clases empleando el lenguaje UML.
- Reconocer y diferenciar los tipos de relaciones que se pueden establecer entre las clases.
- Conocer la manera de representar las diferentes relaciones entre clases empleando el lenguaje UML.
- Distinguir los tipos de clases de análisis de interfaz, de control y de entidad.
- Utilizar herramientas para la elaboración de diagramas de clases.
- Generar código a partir de un diagrama de clases.
- Crear un diagrama de clases a partir de código mediante ingeniería inversa.

Contenidos

- 5.1. Conceptos básicos de la orientación a objetos
- 5.2. El lenguaje UML
- 5.3. Clases, atributos, métodos y visibilidad
- 5.4. Relaciones entre clases
- 5.5. Tipos de clases de análisis
- 5.6. Herramientas para la creación de diagramas de clases
- 5.7. Generación de código a partir de diagramas de clases
- 5.8. Generación de diagramas de clases a partir de código (ingeniería inversa)

Introducción

Para crear una aplicación informática, es necesario llevar a cabo una serie de tareas que ya fueron descritas en las unidades previas. Durante las etapas de análisis y diseño, se deben obtener modelos gráficos que representen la información que es necesario manejar en la aplicación y las operaciones que se deben realizar sobre dicha información. La creación de estos modelos se debe realizar siguiendo ciertas normas y el estándar a nivel internacional para ello es el lenguaje UML (*unified modeling language*).

Una aplicación orientada a objetos consta de una serie de clases relacionadas entre sí, a partir de las cuales, se crean objetos que se envían mensajes. Con relación a ello, el lenguaje UML establece las normas para crear muchos tipos diferentes de diagramas de los cuales los más relevantes son los diagramas de clases, a los que se dedica esta unidad. Así, se estudiarán cómo se representan las clases y sus relaciones siguiendo el lenguaje UML y cómo crear estos diagramas empleando varias herramientas informáticas. También se verá la manera de generar código a partir de un diagrama de clases, y viceversa.

■ 5.1. Conceptos básicos de la orientación a objetos

El **PARADIGMA ORIENTADO A OBJETOS** supuso un cambio de visión en el desarrollo de aplicaciones, en tanto que, **se priorizan los datos a los procesos** a realizar con dicho datos.

Así, las aplicaciones constan de objetos que interactúan entre sí y los **LENGUAJES ORIENTADOS A OBJETOS** deben cumplir una serie de características:

- **Abstracción:** cada objeto funciona como un ente que puede realizar trabajos, informar, cambiar de estado y comunicarse con otros.
- **Encapsulamiento:** de un grupo de elementos formando una entidad.
- **Polimorfismo:** comportamientos diferentes asociados a diferentes objetos pueden recibir el mismo nombre.
- **Herencia:** las clases con atributos o métodos comunes se relacionan formando jerarquías en las que las subclases (clases hijas) heredan atributos y métodos.
- **Modularidad:** la aplicación puede dividirse en partes (módulos).
- **Ocultamiento:** cada objeto está aislado pero puede ser accedido por su interfaz, siempre si se cuentan con derechos adecuados.
- **Recolección de basura:** los objetos sin referencias son eliminados del proceso, liberándose recursos.

■ 5.2. El lenguaje UML

Los principios básicos que se deben seguir a la hora de diseñar el modelo de un sistema son los siguientes:

- La elección acerca de qué modelos crear tiene una enorme influencia sobre cómo se acomete un problema y cómo se da forma a una solución.
- Todo modelo puede ser expresado con diferentes niveles de precisión.
- Los mejores modelos están ligados a la realidad.
- Un único modelo o vista no es suficiente. Cualquier sistema no trivial se aborda mejor mediante un pequeño conjunto de modelos casi independientes y desde múltiples puntos de vista.

El lenguaje UML sigue estos principios. Cabe afirmar que **UML es un lenguaje gráfico que se emplea para visualizar, especificar, construir y documentar un sistema** por medio de diferentes tipos de diagramas que se utilizan en las diferentes tareas de desarrollo del software. Estos modelos se corresponden con diferentes niveles de abstracción y presentan diferente nivel de detalle, o dicho de otro modo, empleando UML se puede estudiar el sistema desde diferentes puntos de vista, ya que no todos los usuarios de un sistema precisan tener la misma visión de este. UML incorpora distintos tipos de diagramas y notaciones gráficas y textuales destinadas a mostrar el sistema desde diferentes perspectivas, que pueden usarse en las diferentes fases del ciclo de vida del desarrollo de software. UML establece las normas que se deben seguir cuando se elaboran estos diagramas.

■ ■ 5.2.1. Tipos de elementos en UML

En el lenguaje UML, existen cuatro tipos de elementos que se describen a continuación

■ ■ Elementos estructurales

En su mayoría, los elementos estructurales son la parte estática de un modelo y representan conceptos o cosas materiales. Son los siguientes:

● **Clase:** conjunto de objetos con atributos comunes y se representan con un rectángulo.

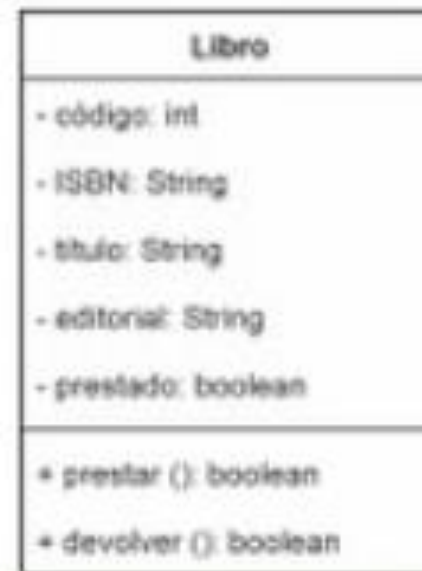


Figura 5.1. Clase llamada Libro con cinco atributos (código, ISBN, título, editorial y prestado) y dos métodos (prestar y devolver).

- **Interfaz:** colección de operaciones que conforman un servicio y se representan con un rectángulo que incluye la etiqueta “interface”.



Figura 5.2. Interfaz IVentana con las operaciones que ofrece (abrir, cerrar, mover y dibujar).

- **Colaboración:** interacción con otros elementos y se representa con una elipse con trazo discontinuo.



Figura 5.3. Representación de una colaboración, en este caso Generar factura.

- **Caso de uso:** describe una secuencia de acciones que ejecuta un sistema y que produce un resultado observable de interés para un usuario/a y se representa con una elipse con trazo continuo.



Figura 5.4. Representación del caso de uso Registrar pedido, que describe un conjunto de operaciones que realiza la aplicación y que son de interés para la persona que la utiliza.

- **Clase activa:** clases cuyos objetos tienen varios hilos de ejecución y se representa con un rectángulo con líneas dobles.

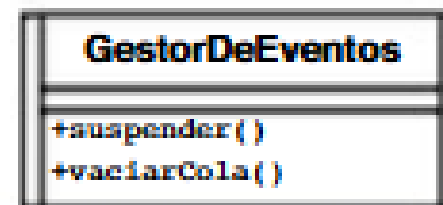


Figura 5.5. Representación de la clase activa GestorDeEventos con los métodos suspender y vaciarCola.

- **Componente:** es una parte modular del diseño y se representa con un icono especial en la parte superior derecha.

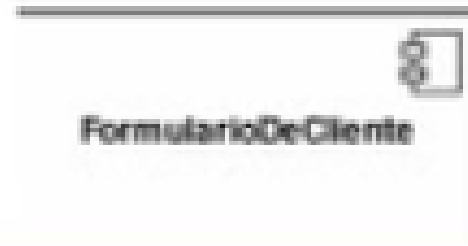


Figura 5.6. Representación del componente FormularioDeCliente.

- **Artefacto:** es un archivo ejecutable, una biblioteca, un script o una base de datos y se representa con un rectángulo con la etiqueta "artifact".

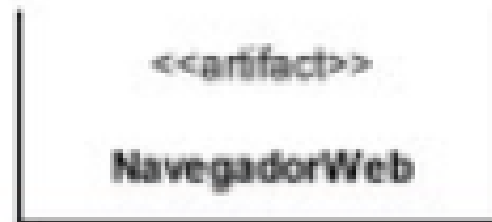


Figura 5.7. Representación de un artefacto llamado NavegadorWeb.

- **Nodo:** es un recurso físico o virtual, generalmente hardware, como servidores, equipos o dispositivos móviles, y se representa con un cubo.

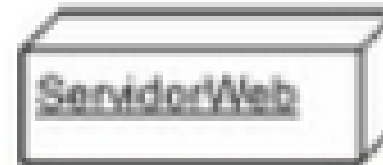


Figura 5.8. Nodo que representa un servidor web.

Elementos de comportamiento

Los elementos de comportamiento son las partes dinámicas de los modelos UML y representan un comportamiento en el tiempo y en el espacio. Suelen estar conectados semánticamente a elementos estructurales. Existen de tres tipos:

● **Interacción:** consiste en un mensaje y se representa con una flecha.



Figura 5.9. Mensaje que solicita la impresión de una factura.

● **Máquina de estados:** especifica el estado en que se encuentra un proceso y se representa con un rectángulo con las esquinas redondeadas.



Figura 5.10. Representación de un estado en espera.

- **Actividad:** *representa una tarea o acción y se representa por un rectángulo con esquinas redondeadas.*



Procesar pedido

Figura 5.11. *Acción que hace referencia al procesamiento de un pedido. Las acciones se representan de igual modo que los estados, pero se distinguen de estos por el contexto en el que se usan.*

Elementos de agrupación

Son las partes organizativas de los modelos UML. El principal elemento de agrupación es el **paquete**. Frente a las clases, que organizan construcciones de implementación, un paquete es un mecanismo de propósito general para organizar el diseño. Un paquete puede incluir elementos estructurales, de comportamiento y otros paquetes. Al contrario de los componentes, que existen en tiempo de ejecución, los paquetes son puramente conceptuales, de manera que solo existen en tiempo de desarrollo. Los paquetes se representan gráficamente como una **carpeta** que lleva en su interior el nombre del paquete (Figura 5.12).

● **Paquete:** es un almacén de otros elementos (estructurales y de comportamiento), incluidos otros paquetes anidados, cuya forma de representación es una carpeta.



Figura 5.12. Representación de un paquete llamado Reglas del negocio.



Elementos de anotación

- **Anotaciones:** son comentarios con la función de describir, clarificar o hacer observaciones y se representan con un rectángulo con una esquina doblada.

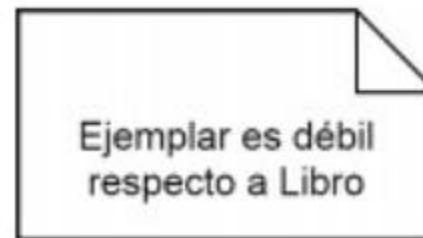


Figura 5.13. Nota que indica que la clase Ejemplar es débil respecto a la clase Libro.

■ ■ 5.2.2. Tipos de diagramas en UML

En UML se pueden construir hasta 14 tipos de diagramas distintos que se pueden agrupar en dos tipos de diagramas genéricos: estructurales y de comportamiento. En los siguientes subapartados, se describen todos los tipos de diagramas y se clasifican según su tipo.

■ ■ ■ Diagramas estructurales

■ ■ ■ Diagramas de comportamiento

Diagramas estructurales

Estos diagramas sirven para visualizar, construir, especificar y documentar los **aspectos estáticos de un sistema**. Se pueden emplear los siete tipos de diagramas estructurales que se indican a continuación:

- **Diagramas de clases:** muestra un conjunto de clases y sus relaciones.

** Un OBJETO es una representación de una instancia de una CLASE en un momento determinado, es una clase en un instante*

- **Diagramas de objetos:** muestra un conjunto de objetos y sus relaciones.
- **Diagramas de componentes:** describen la estructura del software.
- **Diagramas de despliegue:** muestra la distribución de los componentes del software en los nodos de hardware.
- **Diagramas de paquetes:** muestra la estructura de paquetes.
- **Diagramas de perfiles:** es un diagrama especializado para un entorno específicos, como la medicina o la banca, con sus particularidades.
- **Diagramas de estructura compuesta:** permite detallar partes, puertos y conectores. Muestra la estructura interna de un sistema, incluyendo clases, interfaces y sus relaciones, visualizando como interactúan sus componentes.

Diagramas de comportamiento

Estos diagramas sirven para visualizar, especificar, construir y documentar los **aspectos dinámicos de un sistema**. Se pueden distinguir siete tipos de diagramas de comportamiento, si bien los cuatro últimos que se muestran están agrupados bajo el subtipo de diagramas de interacción. Se explicarán en detalle estos diagramas en la Unidad 6.

- **Diagramas de clases:** muestra las interacciones entre actores y sistema.
- **Diagramas de actividades:** representa el **COMPORTAMIENTO** o flujo de trabajo de un sistema, empleando símbolos específicos para describir acciones y decisiones.
- **Diagramas de estado:** muestra los diferentes estados de un objeto y las transiciones entre esos estados a lo largo de su ciclo de vida.
- **Diagramas de interacción:** muestra un conjunto de objetos y los **MENSAJES** que se envían entre ellos.
 - **Diagramas de secuencia:** muestra su secuencia cronológica.
 - **Diagramas de colaboración:** entre un conjunto de objetos.
 - **Diagramas de tiempos:** muestra los tiempos reales de cada uno.
 - **Diagrama global de interacciones:** visión general del flujo integral.

■ 5.3. Clases, atributos, métodos y visibilidad

Un objeto es una instancia de una clase, de modo que una clase está formada por un conjunto de objetos que poseen la misma estructura y el mismo comportamiento. Por ejemplo, la clase *Libro* representa cualquier libro de una biblioteca, siendo un objeto cada uno de los ejemplares de libros disponibles en la biblioteca. Por otro lado, la clase *Cuenta* representa cualquier cuenta bancaria, siendo un objeto cada una de las cuentas de los clientes del banco.

Los atributos son las propiedades que posee una clase. Así, la clase *Libro* podría tener los atributos: código, ISBN, título, editorial y prestado. Cada atributo tendrá un nombre, un tipo de dato (numérico entero, numérico real, cadena de caracteres, etc.) y, posiblemente, un valor por defecto. Cada objeto de la clase *Libro* tendrá estos mismos atributos y cada uno de estos atributos tendrá un valor asignado. La clase *Cuenta* podría tener los atributos número de cuenta y saldo.

Los métodos son las operaciones que se pueden llevar a cabo con los objetos de la clase y definen el comportamiento de la clase. Así, por ejemplo, la clase *Libro* podría disponer de los métodos *prestar* y *devolver* que deben ser invocados cuando un usuario o una usuaria pide prestado un libro de la biblioteca y devuelve un libro que le fue prestado, respectivamente. Por otro lado, la clase *Cuenta* podría disponer de los métodos *ingresarDinero* y *extraerDinero*, que deben ser invocados cuando un cliente ingrese dinero en la cuenta o extraiga dinero de ella, respectivamente. Estos dos métodos recibirán como argumento o parámetro el importe que se desea ingresar y extraer, respectivamente.

La información acerca de las clases se suele representar mediante los llamados diagramas de clases siguiendo la notación UML. Cada clase se representa mediante un rectángulo con tres secciones:

- En la superior se indica el nombre de la clase.
- En la central se coloca información sobre los atributos de la clase.
- En la inferior se coloca información sobre los métodos de la clase.

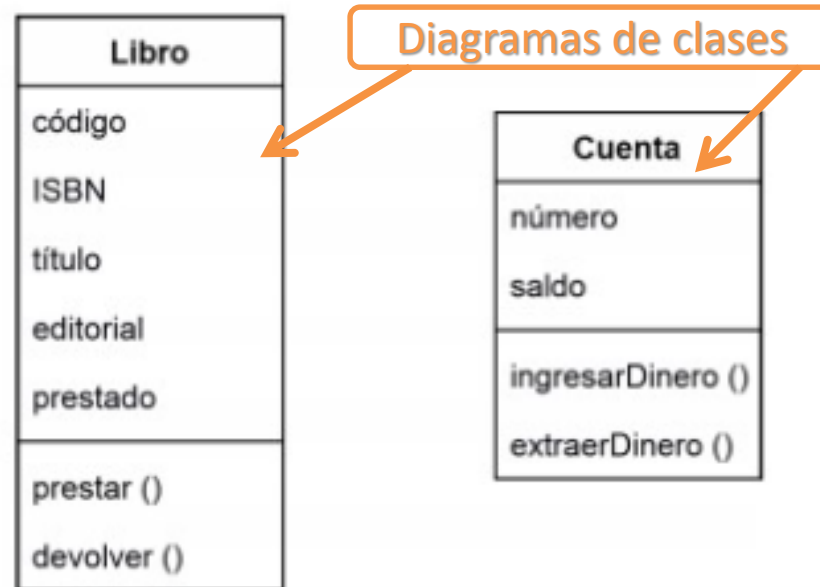


Figura 5.14. Representación de las clases Libro (con los atributos código, ISBN, título, editorial y prestado y los métodos prestar y devolver) y Cuenta (con los atributos número y saldo y los métodos ingresarDinero y extraerDinero).

No obstante, a medida que se avanza en el proceso de desarrollo de software, se puede añadir más información a cada clase. Así, se pueden detallar más los diagramas de clases UML, indicando para cada atributo de una clase un modificador de acceso y el tipo de dato asociado a este, escribiendo:

Modificadores de acceso:

- **Private** si un atributo o un método de una clase se define con el modificador de acceso *private*, quiere decir que ese atributo o método solo es accesible desde métodos de la clase en la que está declarado el atributo o método. Este modificador se simboliza con un guion (-).
- **Public**: si un atributo o un método de una clase se define con el modificador de acceso *public*, quiere decir que ese atributo o método es accesible desde métodos de cualquier clase. Este modificador se simboliza mediante el signo más (+).
- **Protected**: este modificador de acceso indica que el atributo o el método que lo tenga asignado es accesible desde métodos de la propia clase y desde métodos de su subclase o de sus subclases. Este modificador se simboliza por medio del símbolo almohadilla (#).
- **Package**: indica que el atributo o el método es visible en las clases incluidas en el mismo paquete. Este modificador se simboliza con el carácter tilde (~).

Tipos de dato:

- ***int* o *integer*:** hace referencia a un número entero, es decir, sin decimales.
- ***String*:** hace referencia a cadenas de caracteres.
- ***boolean*:** permite almacenar solo los valores *true* (verdadero) o *false* (falso).
- ***float* o *double*:** hace referencia a un número real, es decir, con decimales o bien un número que permite un rango de valores mayor que el tipo *int*.
- ***char*:** permite almacenar un solo carácter, que puede ser una letra, un dígito, un carácter especial (*, @, \$, :, etc.) o una secuencia de escape (como, por ejemplo, el carácter de nueva línea, \n).
- ***Date*:** permite almacenar una fecha.
- ***Time*:** permite almacenar una hora.
- ***DateTime*:** permite almacenar una fecha y una hora conjuntamente.

Además, en el caso de los métodos, se puede indicar el tipo del valor devuelto, si es que el método devuelve algún valor, y para cada uno de los datos que recibe el método (parámetro), el nombre del parámetro y su tipo de dato, siguiendo la siguiente sintaxis:

```
modificador método(parám1: tipoDato1, parám2: tipoDato2, ...): tipoDatoDevuelto
```

■ 5.4. Relaciones entre clases

Para todos los diagramas de clases, se han de establecer relaciones entre las clases, de manera similar a como se establecen relaciones entre las entidades de un diagrama entidad-relación. Existen diferentes tipos de relaciones entre clases, que se van a exponer a continuación.

■ ■ 5.4.1. Agregación

Una **agregación** es un tipo de relación entre clases que representa un objeto compuesto por otros objetos. En esta relación, se distingue la parte que representa el *todo*, que recibe el nombre de **compuesto**, y las diferentes *partes* que lo componen, que reciben el nombre de **componente**.

La agregación implica que solo puede existir una instancia del objeto agregado si existe al menos una instancia relacionada de alguno de los componentes. Por ejemplo, no tiene sentido hablar de un plan de estudios si no tiene al menos una asignatura.

Las relaciones de agregación se representan uniendo el compuesto con sus partes componentes mediante una línea colocando **un rombo con fondo blanco** al lado del compuesto, como se muestra en la Figura 5.16.

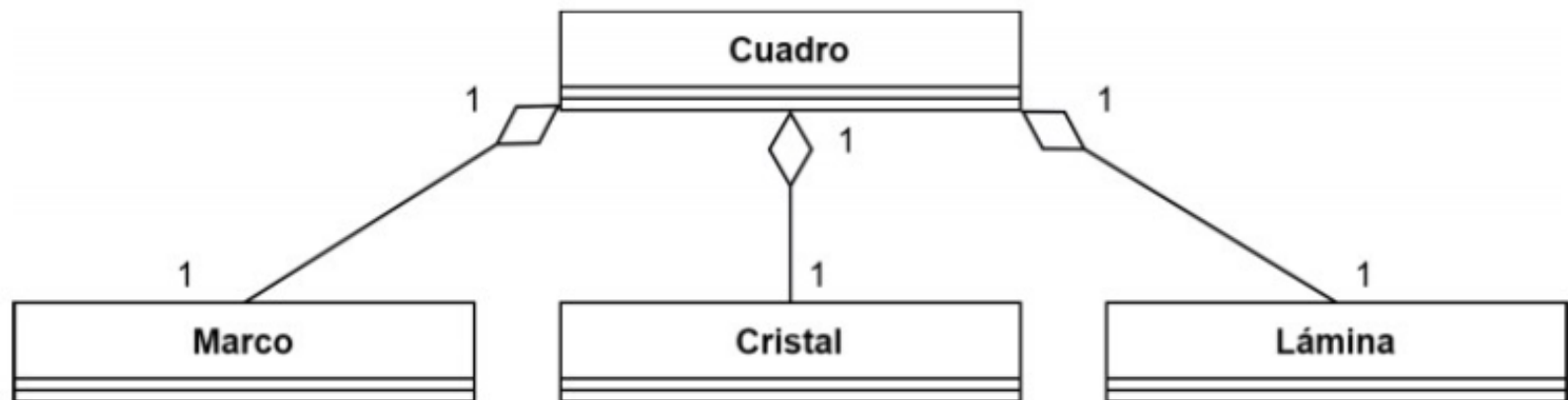


Figura 5.16. Diagrama UML que muestra una relación de agregación entre la clase de tipo compuesto Cuadro y sus partes o componentes Marco, Cristal y Lámina.

Los números 1 al lado de las clases reciben el nombre de **multiplicidades** o **cardinalidades** e indican el **número de objetos de una clase que pueden estar vinculados** con cada objeto de la otra clase. Las cardinalidades en un diagrama UML pueden ser:

Un cuadro, un marco; un cuadro, un cristal; un cuadro, una lámina.

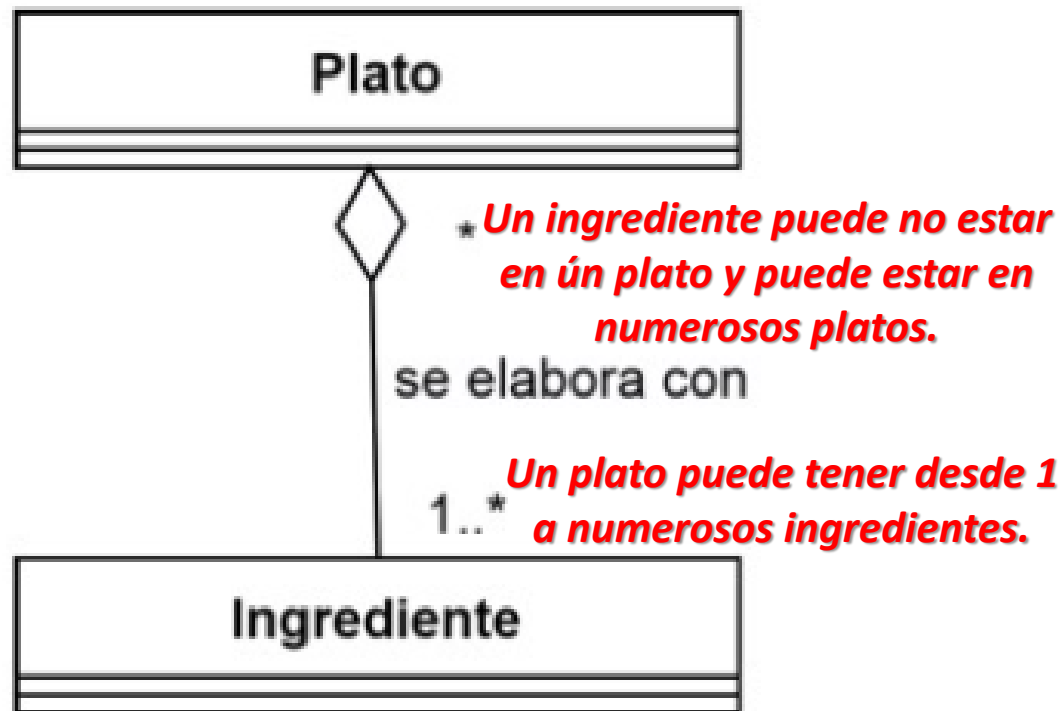


Figura 5.17. Diagrama UML que muestra una relación de agregación entre la clase de tipo compuesto Plato y sus ingredientes.

- 1: uno.
- 0..1: cero o uno.
- 0..*: cero o varios. También se puede representar solo con el símbolo del asterisco (*).
- 1..*: uno o varios.
- N: el número indicado.
- N..M: entre los números N y M.

5.4.2. Composición

La agregación fuerte o **composición es un tipo especial de agregación** en la que la multiplicidad al lado de la clase que representa el compuesto es siempre 1 y en la que la existencia de los componentes depende del compuesto. De esto se puede deducir que, a diferencia de la agregación, la composición impone dos restricciones:

1. Cada componente solo puede estar presente en un compuesto.
2. Si se elimina el compuesto, hay que eliminar todos los componentes vinculados a él.

** La relación de agregación es denominada **RELACIÓN DÉBIL**, mientras que la de composición se denomina **RELACIÓN FUERTE**.*



Figura 5.18. Diagrama UML que muestra una relación de composición entre la clase 'compuesto' PlanEstudios y sus asignaturas.

Prácticas:

Actividad de identificación “Agregación vs composición”:

Lee los siguientes casos e indica si se trata de agregación o de composición:

- 1. Un equipo y sus jugadores***
- 2. Un pedido y sus líneas de pedido***
- 3. Una biblioteca y sus libros***
- 4. Una casa y sus habitaciones***

Actividad “Dibujar diagrama UML”:

Crea diagrama de clases para el siguiente escenario:

Universidad:

Universidad tiene Facultades

Facultades tienen Profesores

Soluciones:

Actividad de identificación “Agregación vs composición”:

Lee los siguientes casos e indica si se trata de agregación o de composición:

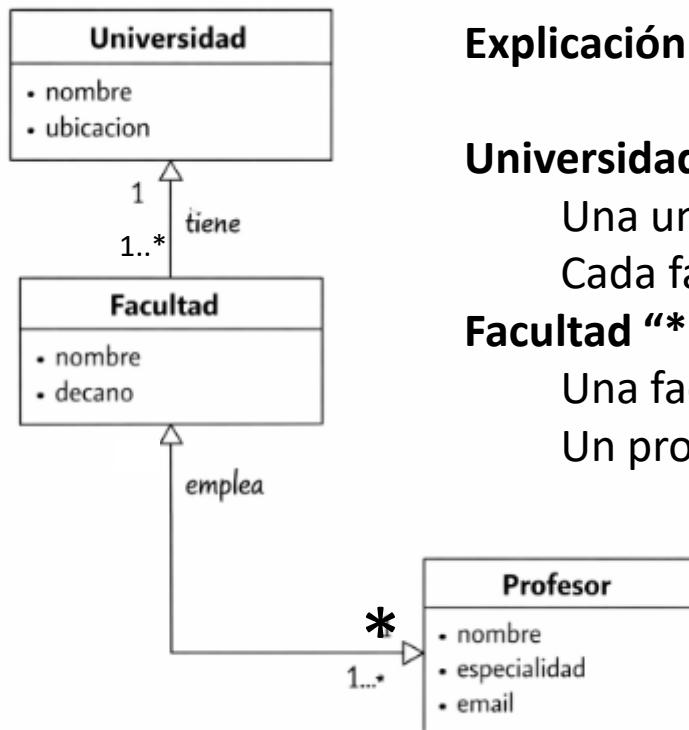
1. *Un equipo y sus jugadores*
2. *Un pedido y sus líneas de pedido*
3. *Una biblioteca y sus libros*
4. *Una casa y sus habitaciones*

1. *Equipo – Jugadores → **AGREGACIÓN***
(Los jugadores pueden existir sin el equipo y cambiar de equipo).
2. *Pedido – Líneas de pedido → **COMPOSICIÓN***
(Si se elimina el pedido, desaparecen sus líneas).
3. *Biblioteca – Libros → **AGREGACIÓN***
(Los libros existen independientemente).
4. *Casa – Habitaciones → **COMPOSICIÓN***
(Las habitaciones no tienen sentido sin la casa).

Soluciones:

Actividad “Dibujar diagrama UML”:

Crea diagrama de clases para el siguiente escenario:



Explicación de las asociaciones:

Universidad "1" -- "1..*" Facultad

Una universidad tiene **una o muchas facultades**.

Cada facultad pertenece a **una sola universidad**.

Facultad "*" -- "1..*" Profesor

Una facultad tiene **uno o muchos profesores**.

Un profesor puede pertenecer a varias facultades.

5.4.3. Generalización y especialización

Las relaciones de generalización-especialización ocurren cuando se establece una jerarquía de clases y subclases motivada por el hecho de que **hay varias clases que poseen atributos y/o métodos comunes.** En este caso, se deben asignar los atributos y métodos comunes a la superclase, dejando los atributos y métodos específicos en las subclases. En este contexto, se habla de **generalización** porque se puede decir que las subclases se generalizan en una superclase, dado que los atributos y métodos comunes a las subclases se asignan a la superclase. Por otro lado, se habla de **especialización** porque la superclase se especializa en una o varias subclases, las cuales poseen atributos y/o métodos específicos además de los de la superclase.

Este tipo de relaciones se representan uniendo mediante una línea cada subclase con su superclase y esta línea lleva en el extremo un triángulo apuntando a la superclase, como se puede observar en la Figura 5.20.

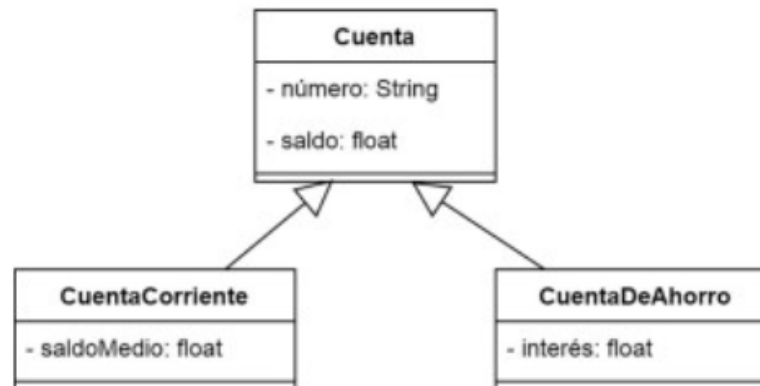


Figura 5.20. Diagrama UML que muestra una relación de generalización-especialización entre la superclase Clase y sus subclases CuentaCorriente y CuentaDeAhorro.

Argot técnico



El hecho de emplear el término **herencia** para referirse al hecho de que las subclases poseen, además de sus atributos y métodos propios, los de la superclase con la que se vinculan, está directamente relacionado con el hecho de que a la superclase se le llame también **clase padre**, y a las subclases, **clases hijas**.

■ ■ 5.4.4. Asociación

Entre las clases pueden existir **relaciones genéricas** que reciben el nombre de *asociaciones*. Cabe definir las **asociaciones** como relaciones entre clases del dominio del problema **que no tienen las características de la agregación ni de la relación de generalización-especialización**. También es común referirse a las asociaciones con el nombre genérico de **relación**, por lo que se emplearán ambos términos indistintamente.

Las asociaciones pueden ser de diferentes tipos en función del número de clases que vinculan. El número de clases que se vinculan por medio de una asociación recibe el nombre de **grado de la relación**. En función de su grado, existen los siguientes tipos de relaciones:

- **Reflexivas:** son las relaciones de grado 1, es decir, aquellas que vinculan a una clase consigo misma.
- **Binarias:** son las relaciones de grado 2, esto es, aquellas que vinculan dos clases.
- **Ternarias, cuaternarias...:** son las relaciones de grado 3, 4..., es decir, aquellas que vinculan tres, cuatro... clases, respectivamente.

Una **asociación reflexiva** se representa por medio de una línea que une una clase consigo misma. Por ejemplo, en una aplicación puede haber una clase *Empleado* y se podría querer reflejar la jerarquía de la plantilla de la empresa, esto es, se podría querer saber, por cada persona empleada, qué empleado o empleada es su jefe directo, que será solamente uno, en caso de tenerlo. Para determinar las cardinalidades o multiplicidades, habría que plantearse las siguientes cuestiones:

- Un empleado tiene como jefe a ningún empleado (si es el máximo responsable de la empresa) o a solo uno (su jefe directo), por lo que la otra cardinalidad será 0..1.

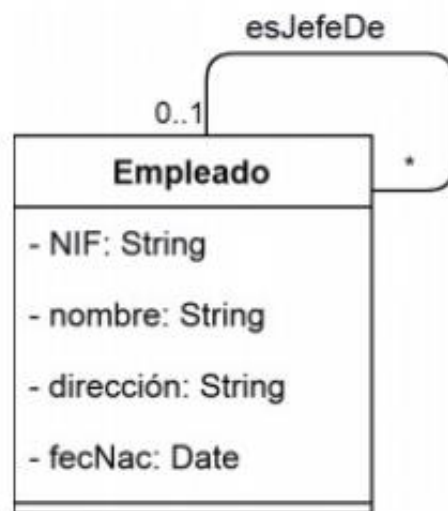


Figura 5.21. Diagrama UML que muestra una relación reflexiva de *Empleado*, la cual refleja el jefe directo de cada persona empleada, en caso de que lo tenga.

Una **relación binaria** se representa por medio de una línea que une las dos clases. Por ejemplo, puede haber una relación entre *Cliente* y *Pedido*, que indique por cada cliente los pedidos que este ha realizado. En este caso, para la determinación de las cardinalidades, habría que plantearse lo siguiente:

- Un cliente puede realizar uno o varios pedidos, por lo que se pondrá 1..* al lado de la clase *Pedido*.
- Un pedido es realizado por un único cliente, por lo que se escribirá 1 al lado de la clase *Cliente*.



Figura 5.22. Diagrama UML que muestra una asociación binaria entre *Cliente* y *Pedido*, que refleja los pedidos que realiza cada cliente.

Navegabilidad

Para las asociaciones, agregaciones y composiciones, también se puede indicar su **navegabilidad**, esto es, **el sentido en el que se puede acceder a información** desde una de las clases intervinientes en la asociación. La navegabilidad se representa mediante una punta de flecha que indica en qué sentido es navegable la relación. Así, en el diagrama que se muestra en la Figura 5.23, en el que se ha dibujado una punta de flecha al lado de la clase *Pedido*, se indica que la relación es navegable de *Cliente* a *Pedido*, lo que quiere decir que a partir de un cliente se puede acceder a todos sus pedidos, pero no es posible saber por cada pedido qué cliente lo ha realizado.



Figura 5.23. Diagrama UML que muestra una asociación binaria entre *Cliente* y *Pedido* navegable solo de *Cliente* a *Pedido*, lo que quiere decir que a partir de un cliente se puede acceder a todos los pedidos que este ha realizado.

Las relaciones de grado mayor que 2 (ternarias, cuaternarias, etc.) se representan mediante un rombo que une a las clases relacionadas. Sería posible tener, por ejemplo, una relación ternaria entre las clases *Atleta*, *Prueba* y *Fecha*, que indica, para los participantes en una competición de atletismo, qué día o días han participado en cada prueba. Para la determinación de las cardinalidades en el caso de las relaciones ternarias, se debe tomar un objeto de cada dos clases y sería necesario plantearse con cuántos objetos de la otra clase como mínimo y como máximo puede estar relacionado ese par de objetos. En este caso, habría que plantearse lo siguiente:

En la Figura 5.24 se muestra el diagrama UML que representa la relación ternaria arriba descrita.

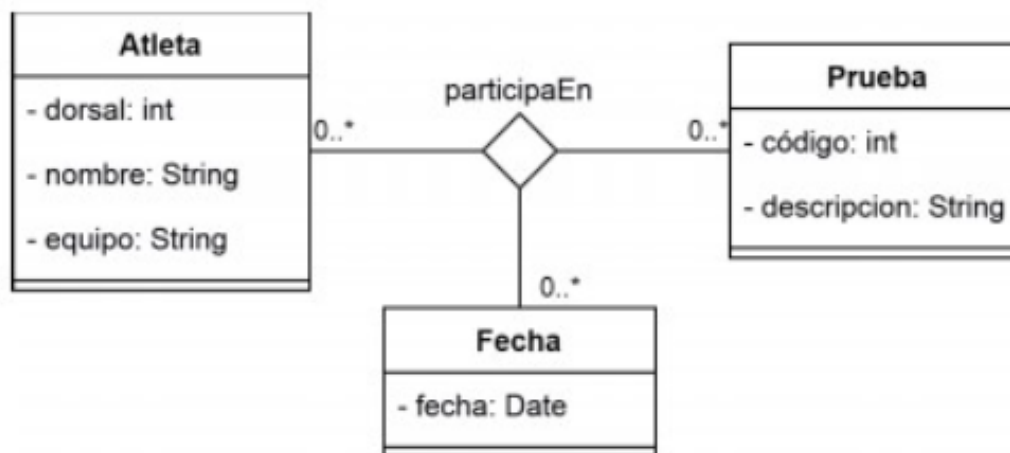


Figura 5.24. Diagrama UML que muestra una asociación ternaria entre *Prueba*, *Atleta* y *Fecha*, que refleja el día o los días en los que participa cada atleta en cada prueba.

5.4.5. Realización

Una relación de **realización** es la que se establece entre una clase *Interface* y las clases que implementan esa interfaz. Una clase *Interface* es un mecanismo que se puede emplear en Java para permitir la herencia múltiple, es decir, que una clase herede de varias superclases. El motivo es que, en Java, una clase solo puede heredar de una superclase, pero puede implementar una o varias interfaces.

Las interfaces **suelen contener métodos comunes a varias clases**, de forma que todas las clases que implementan una interfaz o que establecen una relación de realización con una interfaz, *heredan* de dicha interfaz todos sus métodos.

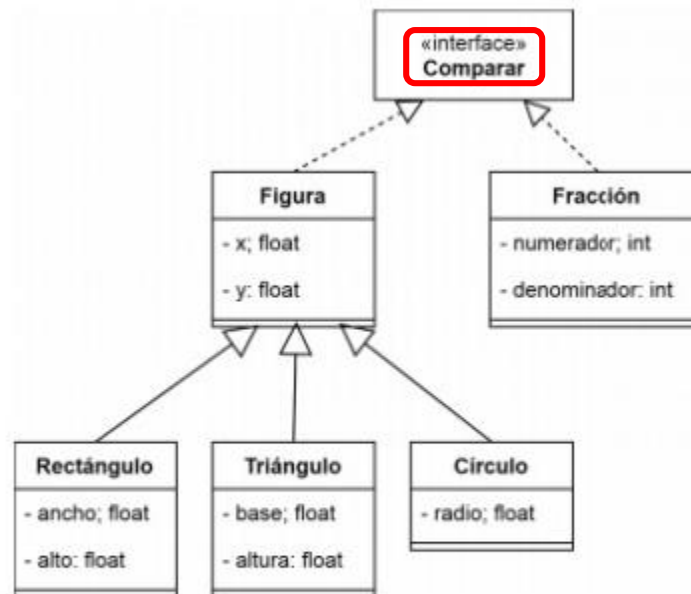


Figura 5.29. Diagrama UML que muestra una relación de realización entre Figura y la interfaz Comparar y otra relación de realización entre Fracción y la interfaz Comparar. Además, la clase Figura es la superclase de las subclases Rectángulo, Triángulo y Círculo.

Actividad propuesta 5.1

Diagrama de clases para una cadena de supermercados I

Crea un diagrama de clases que refleje la información necesaria para gestionar una cadena de supermercados, de acuerdo con los siguientes requisitos:

- La cadena dispone de varios supermercados y, para cada uno de estos, se desea registrar su código, nombre, dirección, número de teléfono y la ciudad donde está ubicado.
- Cada ciudad tiene asignado un código, su nombre y el nombre de la provincia a la que pertenece.
- Se sabe que en una ciudad puede haber varios supermercados de la cadena.

Actividad propuesta 5.1



Figura 5.1. Diagrama de clases que refleja los supermercados de la cadena y las ciudades donde estos están ubicados.

Actividad propuesta 5.2

Diagrama de clases para una cadena de supermercados II

Amplía el diagrama de clases de la Actividad propuesta 5.1 teniendo en cuenta que ahora, por cada supermercado, además se desean conocer los artículos que tiene a la venta. Para ello, se deben considerar los siguientes requisitos:

- Por cada artículo, se desea registrar su código, descripción, precio de venta al público (PVP) y *stock* o existencias que hay de ese artículo en cada supermercado.
- No es necesario que en todos los supermercados se vendan todos los artículos.
- El PVP de un artículo determinado puede diferir de un supermercado a otro, esto es, un mismo artículo no tiene por qué venderse al mismo precio en todos los supermercados.

Actividad propuesta 5.2

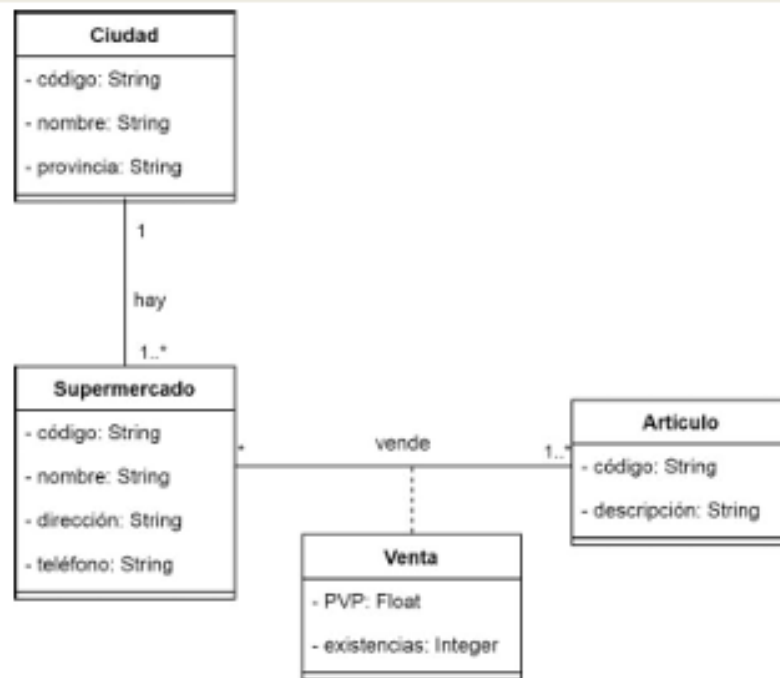


Figura 5.2. Diagramas de clases ampliado en relación con el de la Actividad Propuesta 5.1 que refleja los artículos que se venden en cada supermercado.

Actividad propuesta 5.3

Diagrama de clases para una cadena de supermercados III

Amplía el diagrama de clases de la Actividad propuesta 5.2, teniendo en cuenta que se desea manejar información sobre los proveedores que suministran artículos a los supermercados. Para ello, se debe tener presente que:

- Por cada proveedor, se desea registrar su número de identificación, nombre, dirección y qué artículos suministra a cada supermercado, dado que un proveedor no tiene por qué suministrar los mismos artículos a todos los supermercados de la cadena.
- Además, un mismo artículo lo puede vender un proveedor a diferentes precios en función del supermercado al que se lo suministra.
- Se quiere tener constancia de qué artículos suministra cada proveedor a cada supermercado y a qué precio. Este se conoce habitualmente como *precio de venta de distribución* (PVD).

Actividad propuesta 5.3

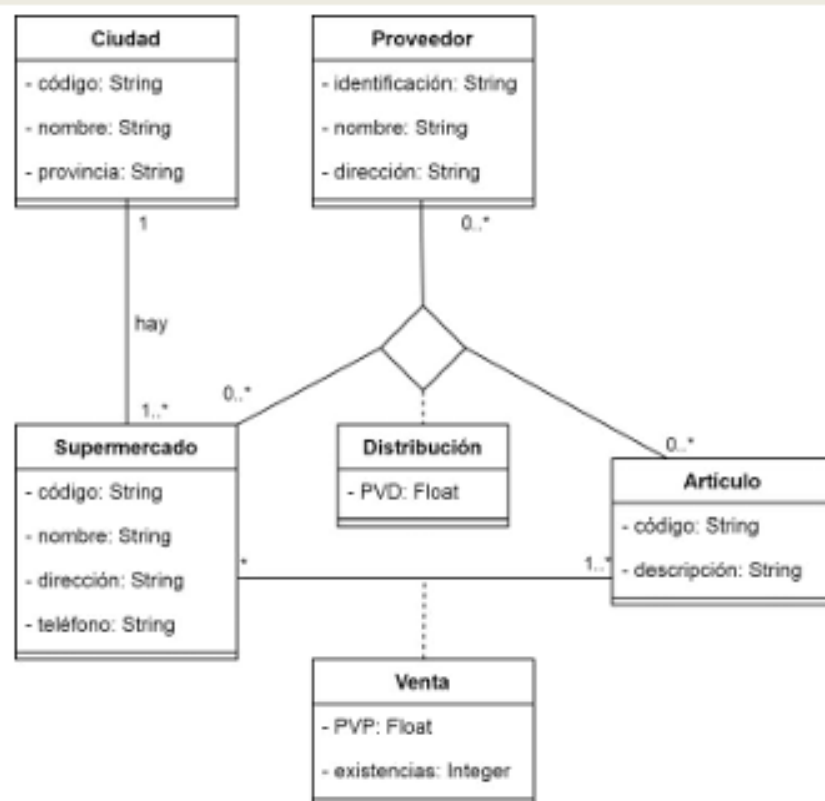


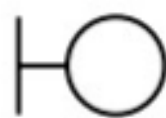
Figura 5.3. Diagramas de clases ampliado en relación con el de la Actividad Propuesta 5.2 que refleja información sobre los proveedores que suministran los artículos que se venden en cada supermercado.

■ 5.5. Tipos de clases de análisis

Durante la fase de análisis del desarrollo de una aplicación, siguiendo el paradigma orientado a objetos, se deben identificar tres tipos de clases:

1. Las **clases de interfaz** se usan para modelar la interacción entre el sistema y sus actores. Esta interacción consiste normalmente en recibir y mostrar información o pedirla a las personas usuarias o sistemas externos. Estas clases se deben limitar a describir la información y peticiones que se intercambian los actores y el sistema a través de ellas. Cada clase de interfaz se debería asociar a, al menos, un actor, y viceversa. Se usan estas clases para modelar ventanas, formularios, impresos, etc.
2. Las **clases de entidad** se emplean para modelar información que persiste en el tiempo. Suelen mostrar una estructura de datos lógica.
3. Las **clases de control** realizan la coordinación y el control sobre otros objetos del sistema. Toda la dinámica del sistema es modelada por clases de control. Se asocia una clase de control a cada aplicación y esta clase modela la dinámica del sistema delegando trabajo a otras clases.

Cada tipo de clase se representa como se muestra en la Figura 5.30.



Clase de interfaz



Clase de control



Clase de entidad

Figura 5.30. Representación de los tipos de clases de interfaz, de control y de entidad.

■ 5.6. Herramientas para la creación de diagramas de clases

En la actualidad, existen muchas herramientas para la creación de diagramas de clases. En esta unidad, se aprenderá a emplear tres de ellas:

- **diagrams.net:** es una herramienta muy sencilla de aprender y utilizar, pero que no realiza comprobaciones sobre los diagramas creados, por lo que es posible crear diagramas que no respeten las normas de UML.
- **Papyrus:** es una herramienta mucho más avanzada que se puede integrar en Eclipse como un módulo y que permite realizar todo tipo de diagramas UML.
- **Modello:** es una aplicación de modelado UML de código abierto que permite crear todo tipo de diagramas y, además, generar el código correspondiente.

5.19. Realiza un diagrama de clases sin métodos para una empresa dedicada al alquiler de automóviles, teniendo en cuenta los siguientes requisitos:

- Un determinado cliente puede tener, en un momento dado, varias reservas hechas.
- De cada cliente, se desea almacenar su DNI, nombre, dirección y teléfono. Además, los clientes se diferencian por un código único.
- Cada cliente puede ser avalado por otro cliente de la empresa. Un cliente puede avalar a varios.
- Una reserva la realiza un único cliente, pero puede involucrar varios coches.
- Es importante registrar la fecha de inicio y fin de la reserva, el precio de alquiler de cada uno de los coches, los litros de gasolina en el momento de realizar la reserva, el precio total de la reserva y un indicador que informe acerca de si cada coche ha sido entregado o no.
- Todo coche tiene siempre asignado un determinado garaje. De cada coche, se requiere registrar su matrícula, modelo, color y marca.
- Cada reserva se realiza en una determinada agencia. De cada agencia, se requiere registrar su nombre y dirección.
- Sobre cada garaje, es necesario conocer su código único, ubicación y capacidad o número máximo de coches que caben en el garaje.

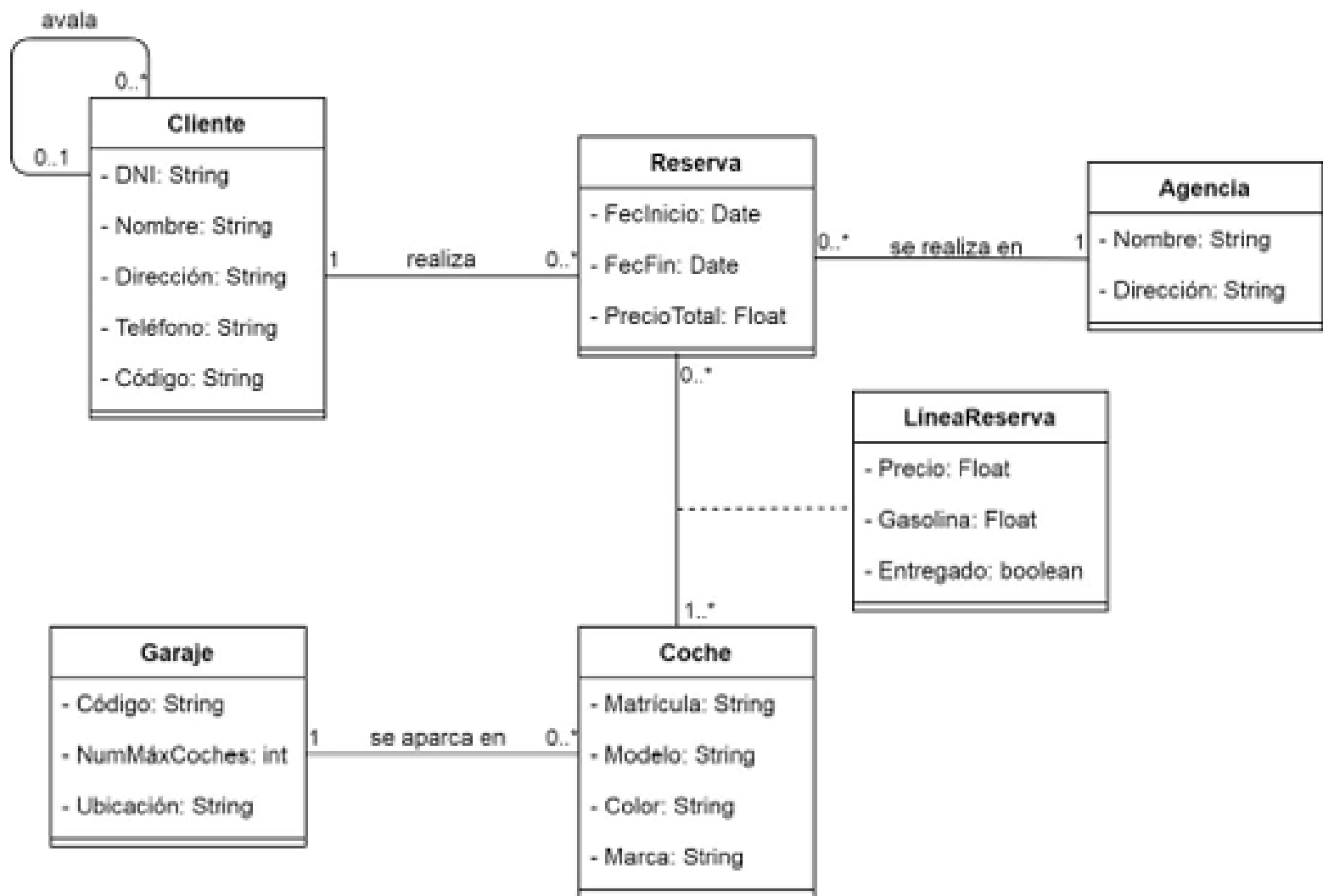


Figura 5.4. Diagrama de clases para una empresa dedicada al alquiler de vehículos.

5.20. Elabora un diagrama de clases sin métodos para un sistema de monitorización de sensores, teniendo en cuenta los siguientes requisitos:

- El sistema monitoriza sensores y también informa sobre posibles problemas.
- Cada sensor lo describe su fabricante y número de modelo, secuencia de iniciación (que se envía al sensor para iniciarlo), factor de escala y unidad de medida, intervalo de muestreo, ubicación, estado (encendido, apagado, en reserva), valor actual y umbral de alarma. Hay un tipo especial de sensores, que son los *sensores críticos*. Estos tienen una característica adicional, que es su tolerancia del intervalo de muestreo.
- Los sensores están instalados en edificios. El sistema hace un seguimiento de los sensores de cada edificio. De cada edificio, se conoce su dirección y el número de contacto en caso de emergencia.
- El sistema activa determinados dispositivos de alarma siempre que se supera el umbral de un sensor. Los dispositivos de alarma tienen dos características de las cuales depende su funcionamiento: el estado del dispositivo y la duración de la señal de alarma.
- El sistema registra información de la fecha, hora, gravedad, tiempo de reparación y estado de cada suceso de alarma.

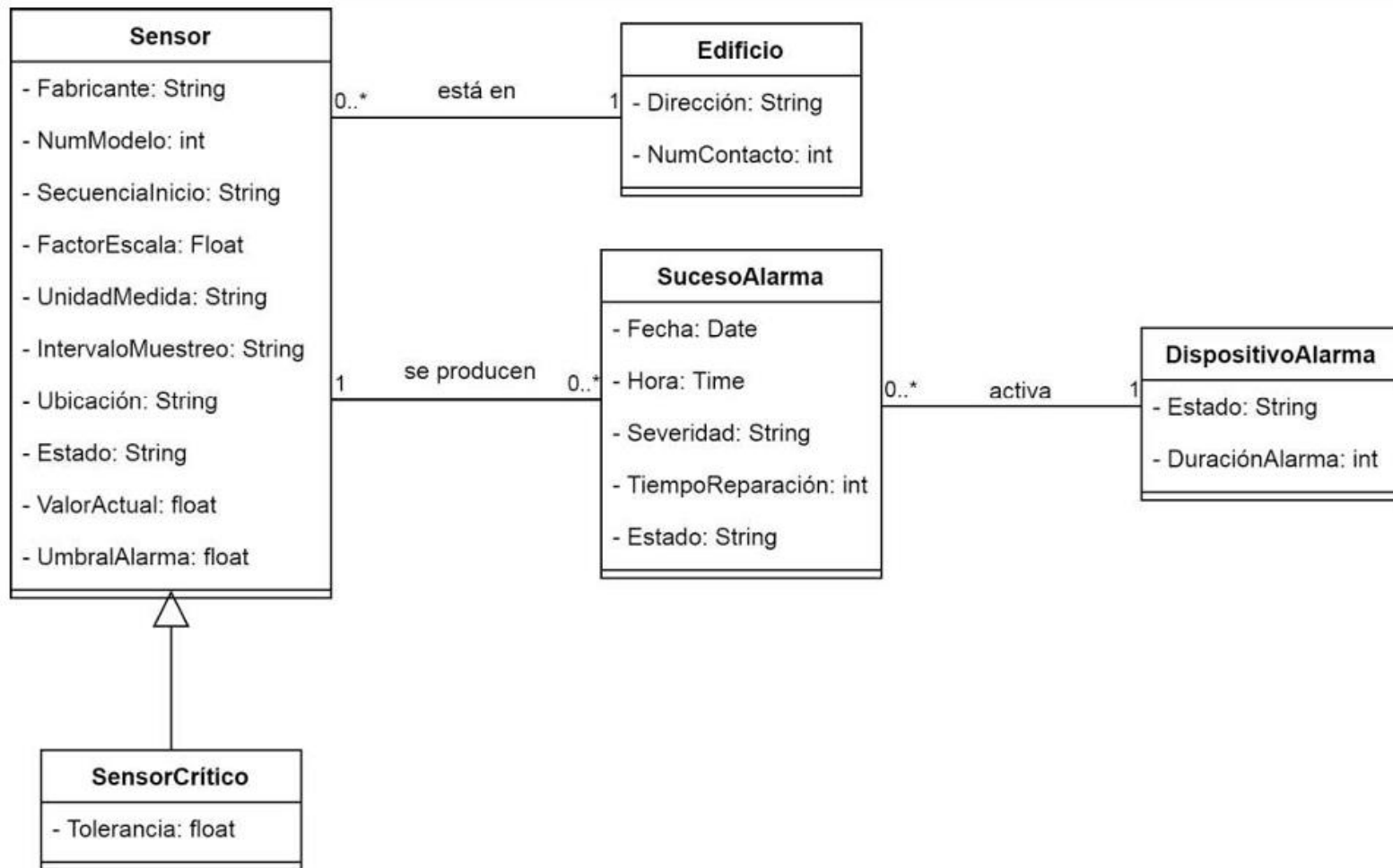


Figura 5.5. Diagrama de clases para un sistema de monitorización de sensores.

5.21. Realiza un diagrama de clases sin métodos para una entidad bancaria, de la que es necesario saber su CIF, nombre y dirección. Tras una entrevista se ha obtenido la siguiente información:

- *Listado de sucursales:* código de la sucursal, dirección y teléfono.
- *Listado de cuentas corrientes:* número de la cuenta, saldo y, además, NIF, nombre y apellidos y dirección de las personas que pueden utilizar esa cuenta.
- *Listado de operaciones realizables:* código de la operación y descripción.
- *Listado de recibos domiciliados:* número de recibo, número de cuenta de pago, importe del recibo, entidad emisora (empresa que emite el recibo) y fecha.

Las reglas que regulan este sistema son las siguientes:

- El banco tiene muchas sucursales.
- Cada sucursal tiene una serie de cuentas corrientes.
- Cada cuenta corriente tiene asociados uno o más clientes. Sin embargo, es posible que varios clientes con la misma cuenta (por ejemplo, personas de la misma familia: padres e hijos menores) no puedan realizar las mismas operaciones.
- Cada cliente puede tener varias cuentas. En cada una de ellas puede realizar operaciones distintas.
- Cada cuenta puede o no tener uno o varios recibos domiciliados.

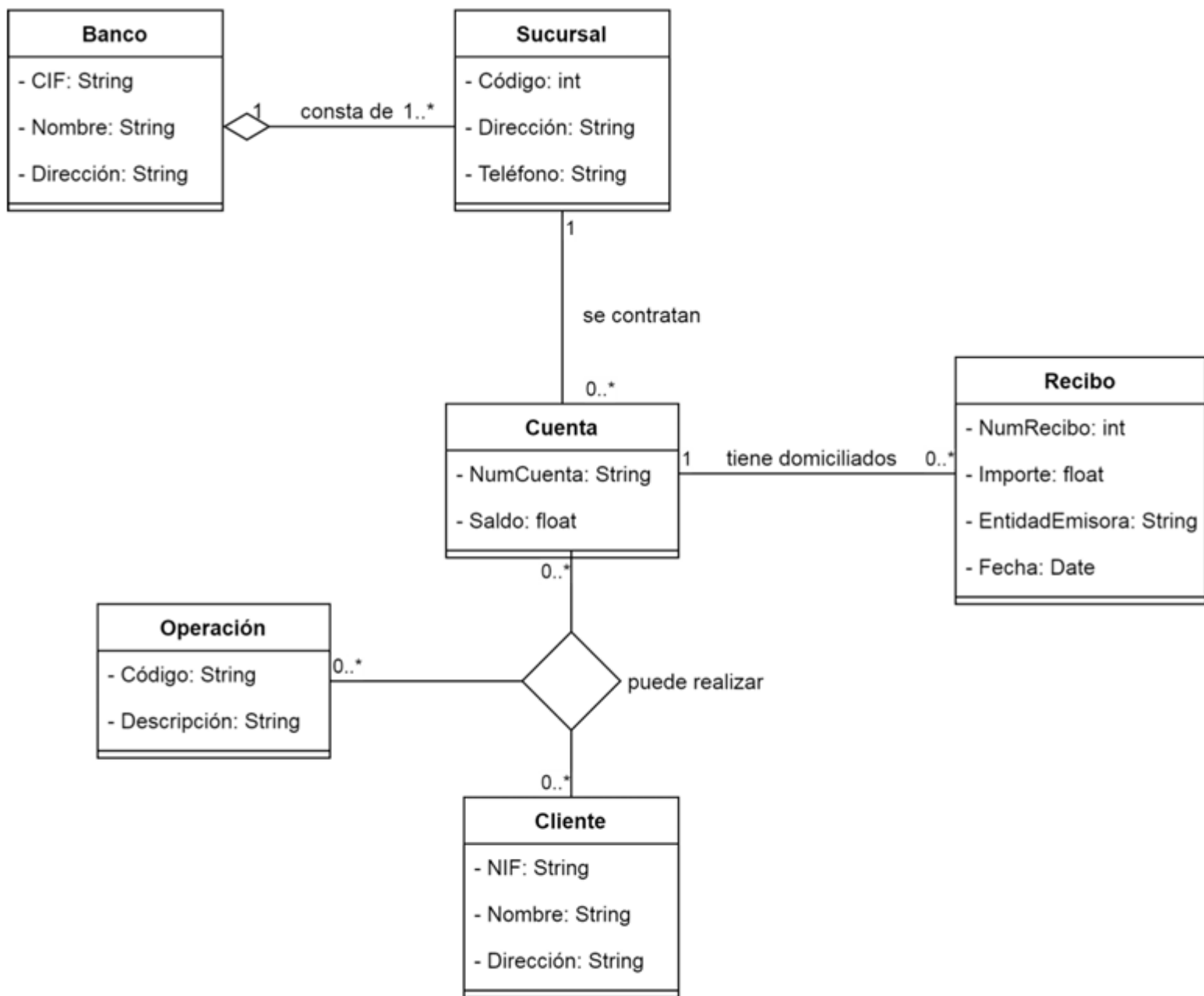


Figura 5.6. Diagrama de clases para una entidad bancaria.

5.22. Elabora un diagrama de clases sin métodos para una empresa que pertenece a un grupo nacional de agencias que se dedican a la venta al por mayor, de acuerdo con las siguientes especificaciones:

- Los clientes de la empresa pueden ser mayoristas, minoristas e, incluso, clientes del propio grupo de empresas. Acerca de todos los clientes, se debe conocer su NIF, nombre y apellidos, dirección y teléfono. Además, para los mayoristas se necesita saber el almacén y el número de tarjeta bancaria. En el caso de los minoristas, interesa el número de cuenta bancaria; y en el caso de clientes del propio grupo, la actividad a la que se dedican.
- Cuando un cliente hace un pedido, al que se asigna un número y del que se registra asimismo su fecha, se genera un documento con la relación de los artículos solicitados. De cada artículo se precisa saber su código, nombre, precio, existencias actuales y el número mínimo y máximo de existencias recomendables, así como el número de unidades de ese artículo solicitadas en el pedido. Cada artículo lleva asociado un tipo de IVA, que puede ser del 4, del 10 o del 21 %. Por otra parte, algunos artículos tienen asociado un envase, del que se desea almacenar su código, descripción y precio.
- El pedido puede ser modificado como consecuencia de devoluciones, siniestros, cambios de última hora en el criterio del cliente, etcétera.
- Al final de cada mes, la empresa pone en marcha el proceso de facturación que consiste en crear una factura por cada cliente, en la que se incluyen los pedidos correspondientes a ese mes. Las facturas se numeran y llevan una fecha asociada.
- En los pedidos intervienen los llamados *vendedores* o *comisionistas*, que son empleados de la empresa cuyo propósito es promocionar y vender los artículos. En cada pedido interviene un solo comisionista. Al final de cada mes, reciben sus liquidaciones correspondientes. Cada vendedor-comisionista tiene un porcentaje sobre el total de sus ventas y una determinada antigüedad. Aparte de estos datos, se precisa registrar su NIF, nombre y apellidos, teléfono y NSS (número de Seguridad Social). Se considera venta solamente si se ha facturado.

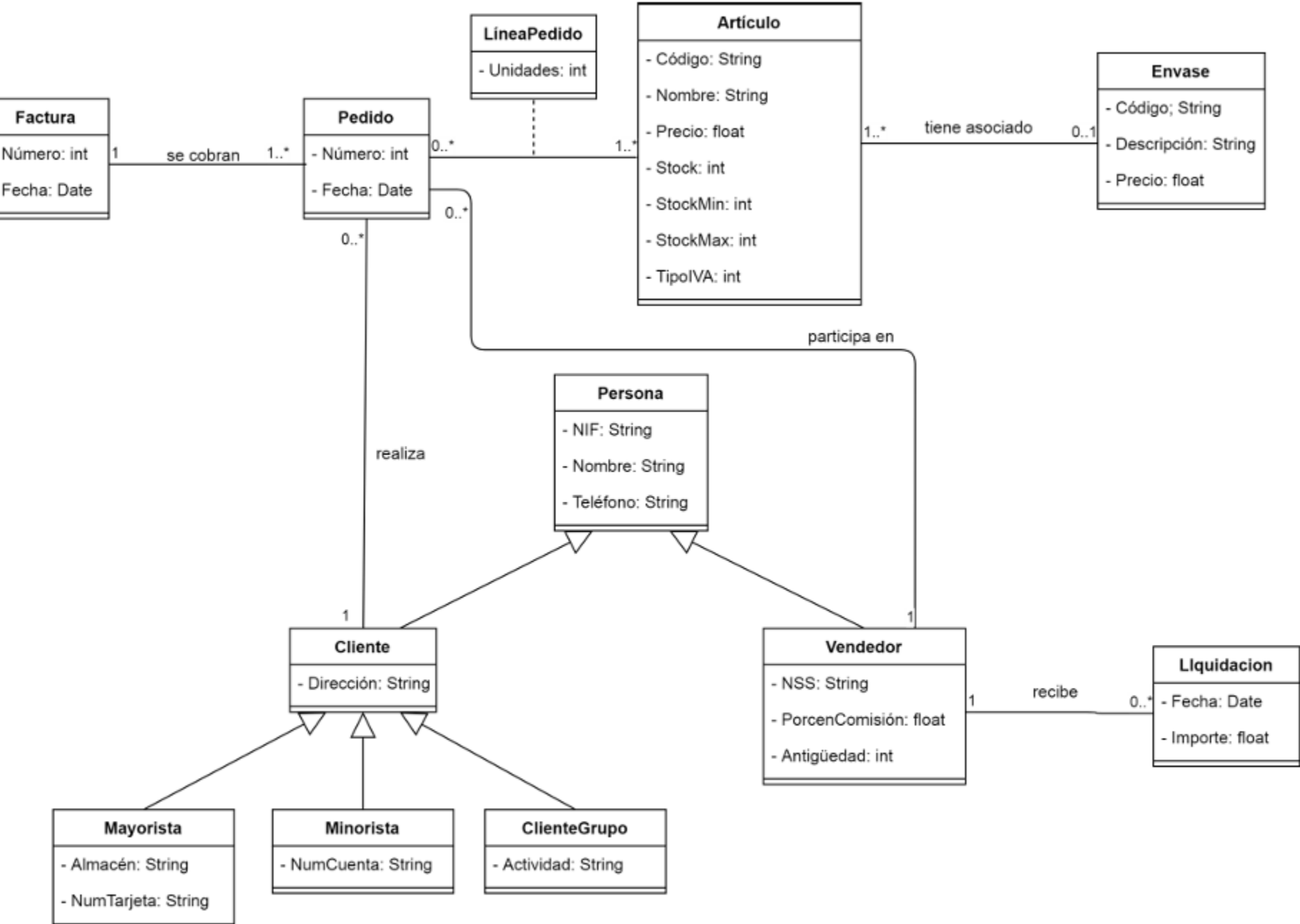


Figura 5.7. Diagrama de clases para una empresa de venta al por mayor.